1 Question 1

First, to find a easily invertible $f^*(x)$, I manipulated a second order polynomial on desmos to find one that matched $h(x)$ well. Visually, I found $f^*(x) = C(4x + 1)$:

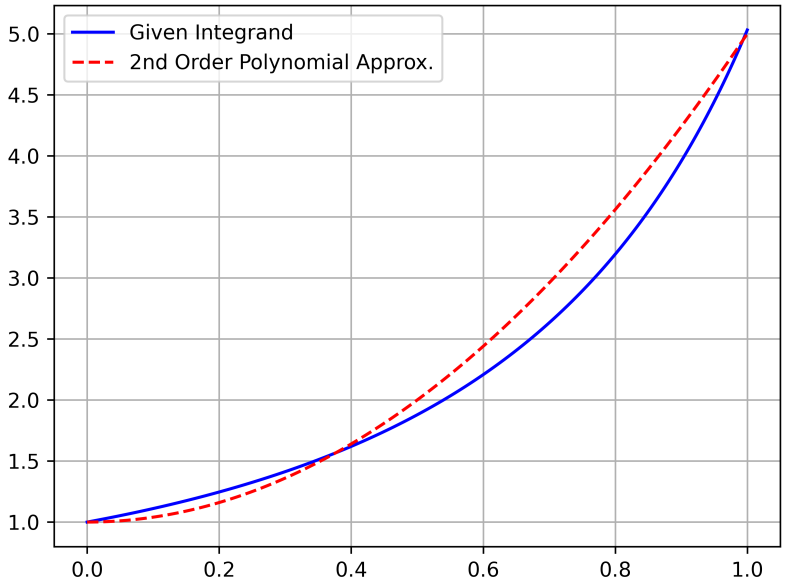


Figure 1: $h(x)$ vs first order polynomial approximation.

I then integrated to find C:

$$\int_0^1 f^*(x) = 1 = C \int_0^1 (4x + 1)dx = C\frac{7}{3} \quad \Rightarrow C = \frac{3}{7} \tag{1}$$

$$f^*(x) = \frac{3}{7}(4x^2 + 1) \tag{2}$$

For probability mixing, need

$$\sum \frac{1}{\alpha_i} = 1 \tag{3}$$

Notice we can rearrange $f^*(x)$:

$$f^*(x) = \frac{4}{7}(3x^2) + \frac{3}{7} \tag{4}$$

We can split $f^*(x)$ into the following

$$f_1 = \frac{4}{7}(3x^2), \quad \alpha_1 = \frac{7}{4} \tag{5}$$

$$f_2 = \frac{3}{7}, \quad \alpha_2 = \frac{7}{3} \tag{6}$$

Apply FFMC for mixed functions:

$$\eta = F_1(x) = \alpha_1 \int_0^1 f_1(x) = x^3, \quad x = \eta^{1/3} \tag{7}$$

$$\eta = F_2(x) = \alpha_2 \int_0^1 f_2(x) = x, \quad x = \eta \tag{8}$$

Which I used for the importance sampling. Such that

$$g^*(x) = \frac{\exp(x)}{\cos(x)} \frac{1}{\frac{4}{7}(3x^2) + \frac{3}{7}} \tag{9}$$

I copied my results and told Google Gemini "make this into a Latex table with a speed-up factor". With the exact same number of samples, if we pick a good $f^*(x)$ for importance sampling, we achieve a massive two orders-of-magnitude speed-up factor.

Table 1: Monte Carlo Variance Reduction Performance: Importance Sampling vs. Crude Baseline

Method	**Expected Value**	**Sample S_x**	**Rel. Var (%)**	**Speedup Factor**
Crude MC	2.21779	1.07227	51.84%	1.00×
Importance Sampling	2.21733	0.13477	0.82%	63.30×

2 Question 2

I took a screenshot of table 5.2 from "MC4_Integrals..." and pasted the results of my code into Google Gemini and said "these are my results from a python script. use the data in these results to generate a Latex table with the same format as the one in the picture I included."

Method	Number of samples	Expected value	sample S_x	Rel. Diff. (%)
Standard				
	100	0.79304	1.64083e-02	2.06904e-02
	500	0.79095	7.47991e-03	9.45687e-03
	1,000	0.78553	5.17515e-03	6.58813e-03
	5,000	0.78728	2.27753e-03	2.89290e-03
	10,000	0.78692	1.61802e-03	2.05614e-03
	50,000	0.78634	7.16932e-04	9.11738e-04
	100,000	0.78500	5.09448e-04	6.48980e-04
Uniform stratified				
	100	0.78413	1.57642e-04	2.01040e-04
	500	0.78518	3.08783e-05	3.93262e-05
	1,000	0.78519	1.53499e-05	1.95492e-05
	5,000	0.78572	3.03263e-06	3.85970e-06
	10,000	0.78550	1.53153e-06	1.94975e-06
	50,000	0.78539	3.06242e-07	3.89923e-07
	100,000	0.78539	1.52805e-07	1.94561e-07
Optimum stratified				
	100	0.78505	2.01553e-04	2.56740e-04
	500	0.78546	2.96322e-05	3.77262e-05
	1,000	0.78520	1.42145e-05	1.81030e-05
	5,000	0.78554	2.91127e-06	3.70606e-06
	10,000	0.78568	1.45829e-06	1.85609e-06
	50,000	0.78541	2.90966e-07	3.70462e-07
	100,000	0.78538	1.45170e-07	1.84841e-07

I want to note the standard deviations S_x for the uniform and optimum stratified samplings were relatively close. The integrand is quite smooth $x \in [0, 1]$, which is why the uniform and stratified samplings performed similarly. However, we could choose an integrand with a larger gradient to capitalize on the optimum sampling better.